

# PostgreSQL Autovacuum — Diagnostic Query Playbook

*Copy-paste queries to inspect, diagnose, and monitor autovacuum — PostgreSQL 14-18.*

**Companion to:** Autovacuum Tuning Expert Reference (v14-18).

**Version notes:** Queries are labelled `([PG17+])` / `([PG18])` where they use columns or features not present in earlier releases. Unlabelled queries run on 14-18. Trim columns that error on older minors.

---

## Contents

- [0. How to use this playbook](#)
  - [1. Health snapshot — is it on and what's running](#)
  - [2. Dead tuples & bloat — where is the churn](#)
  - [3. The xmin horizon — why dead tuples won't die](#)
  - [4. Transaction-ID wraparound & freeze age](#)
  - [5. Live vacuum progress](#)
  - [6. Effective settings — global and per-table](#)
  - [7. Blocking, cancellation & starvation](#)
  - [8. I/O attribution & AIO](#)
  - [9. Live-challenge triage order](#)
  - [Appendix. Common remediation snippets](#)
- 

## 0. How to use this playbook

Work top-down. The fastest way to misdiagnose autovacuum is to jump to GUC tuning before confirming vacuum is even *allowed* to remove dead tuples. The sections are ordered the way you should actually triage an incident:

- **§1 Health snapshot** — is autovacuum on, what's running right now, what's overdue.
- **§2 Dead tuples & bloat** — where is the churn and how far behind is vacuum.
- **§3 The xmin horizon** — the #1 cause of "vacuum runs but dead tuples stay". Check this before tuning anything.
- **§4 Wraparound / freeze age** — `relfrozenxid` / `datfrozenxid` / `multixact` age monitoring.
- **§5 Live progress** — `pg_stat_progress_vacuum`, phases, multi-pass detection.
- **§6 Effective settings** — global GUCs and per-table overrides actually in force.
- **§7 Blocking & cancellation** — locks that cancel autovacuum, anti-wraparound workers.
- **§8 I/O & AIO** — `pg_stat_io` vacuum context, PG18 `pg_aios`.
- **§9 Triage flow** — a one-page decision order for a live challenge.

**CONVENTION** — Replace schema/table placeholders like `(public.orders)` as needed. Queries default to the current database unless they read shared catalogs `(pg_database, pg_replication_slots)`.

---

## 1. Health snapshot — is it on and what's running

### 1.1 Is autovacuum enabled and how is it configured (top-line)

*One glance at the master switches and daemon sizing.*

```

sql

SELECT name, setting, unit, context
FROM   pg_settings
WHERE  name IN (
    'autovacuum',
    'autovacuum_max_workers',
    'autovacuum_worker_slots',          -- PG18
    'autovacuum_naptime',
    'autovacuum_vacuum_cost_limit',
    'autovacuum_vacuum_cost_delay',
    'autovacuum_vacuum_scale_factor',
    'autovacuum_vacuum_threshold',
    'autovacuum_vacuum_insert_scale_factor',
    'autovacuum_vacuum_insert_threshold',
    'autovacuum_vacuum_max_threshold', -- PG18
    'autovacuum_analyze_scale_factor',
    'autovacuum_analyze_threshold',
    'autovacuum_freeze_max_age',
    'autovacuum_multixact_freeze_max_age',
    'autovacuum_work_mem',
    'maintenance_work_mem',
    'vacuum_cost_limit', 'vacuum_cost_delay',
    'vacuum_failsafe_age', 'vacuum_multixact_failsafe_age',
    'log_autovacuum_min_duration')
ORDER BY name;

```

**READ** — `context` tells you how to change each one: `postmaster` = restart, `sighup` = `pg_reload_conf()`, `user`/`superuser` = `SET`. Rows returning nothing are GUCs your version doesn't have (e.g. `worker_slots`/`max_threshold` before PG18).

## 1.2 Autovacuum workers running right now

*What each worker is doing, and for how long.*

```

sql

SELECT pid,
       datname,
       username,
       state,
       wait_event_type,
       wait_event,
       now() - xact_start      AS xact_age,
       now() - query_start    AS running_for,
       query
FROM   pg_stat_activity
WHERE  backend_type = 'autovacuum worker'    -- PG10+
       OR query ILIKE 'autovacuum:%'
ORDER BY xact_start;

```

**READ** — The query text shows the table, and whether it is an anti-wraparound run: look for `((to prevent wraparound))`. Those are NOT auto-cancelled by conflicting locks.

## 1.3 How long since each table was last (auto)vacuumed / analyzed

*Spot tables that never get serviced.*

```

sql

```

```

SELECT schemaname, relname,
       last_vacuum, last_autovacuum,
       last_analyze, last_autoanalyze,
       vacuum_count, autovacuum_count,
       analyze_count, autoanalyze_count
FROM   pg_stat_user_tables
ORDER  BY COALESCE(last_autovacuum, last_vacuum, 'epoch') ASC NULLS FIRST
LIMIT  40;

```

**READ** — NULL (`last_autovacuum`) on a high-write table = it has never triggered.  
 Cross-check with dead-tuple counts in §2 before concluding it's a problem.

---

## 2. Dead tuples & bloat — where is the churn

### 2.1 Dead tuples vs. the autovacuum trigger threshold

The core "is this table overdue?" query. Computes each table's live trigger point from its effective settings.

```

sql

WITH s AS (
  SELECT
    current_setting('autovacuum_vacuum_threshold')::bigint AS base_thr,
    current_setting('autovacuum_vacuum_scale_factor')::float AS scale
)
SELECT
  st.schemaname, st.relname,
  c.reltuples::bigint AS est_rows,
  st.n_dead_tup,
  st.n_live_tup,
  round(s.base_thr + s.scale * c.reltuples) AS av_trigger_at,
  CASE WHEN st.n_dead_tup
    > s.base_thr + s.scale * c.reltuples
    THEN 'OVER THRESHOLD' ELSE 'ok' END AS status,
  st.last_autovacuum
FROM   pg_stat_user_tables st
JOIN   pg_class c ON c.oid = st.relid
CROSS JOIN s
WHERE  c.reltuples > 0
ORDER  BY st.n_dead_tup DESC
LIMIT  40;

```

**READ** — This uses GLOBAL scale/threshold. Tables with per-table storage params (§6.2) have a different real trigger — reconcile the two. On PG18 also cap at (`autovacuum_vacuum_max_threshold`).

### 2.2 Dead-tuple percentage (bloat proxy, no extension needed)

Fast ratio to rank churn without scanning heaps.

```

sql

```

```

SELECT schemaname, relname,
       n_live_tup, n_dead_tup,
       CASE WHEN n_live_tup + n_dead_tup > 0
            THEN round(100.0 * n_dead_tup
                      / (n_live_tup + n_dead_tup), 1)
            ELSE 0 END AS dead_pct,
       n_mod_since_analyze,
       last_autovacuum, last_autoanalyze
FROM   pg_stat_user_tables
WHERE  n_dead_tup > 1000
ORDER  BY dead_pct DESC, n_dead_tup DESC
LIMIT  40;

```

**READ** — `dead_pct` is an estimate from the stats counters, not a heap measurement. For the true figure use 2.3 (`pgstattuple`).

## 2.3 Exact bloat measurement (`pgstattuple`)

*Ground-truth dead space for a suspect table. Heavy — run on one table at a time.*

```

sql

CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT * FROM pgstattuple('public.orders');           -- full heap scan
-- Cheaper sampled variant for very large tables:
SELECT * FROM pgstattuple_approx('public.orders');
-- Index bloat:
SELECT * FROM pgstatindex('public.orders_pkey');

```

**READ** — `dead_tuple_percent` and `free_percent` are the numbers that matter. `pgstattuple_approx` skips the full scan; use it above ~10 GB.

## 2.4 Table + index physical size (context for bloat)

*Put dead-tuple counts in size context.*

```

sql

SELECT schemaname, relname,
       pg_size_pretty(pg_table_size(relid))           AS table_size,
       pg_size_pretty(pg_indexes_size(relid))         AS indexes_size,
       pg_size_pretty(pg_total_relation_size(relid))  AS total_size,
       n_dead_tup
FROM   pg_stat_user_tables
ORDER  BY pg_total_relation_size(relid) DESC
LIMIT  30;

```

# 3. The xmin horizon — why dead tuples won't die

VACUUM can only remove tuples whose deleting transaction precedes the global `xmin` horizon. If the horizon is held back, autovacuum runs, reports "N dead but not yet removable", and bloat grows regardless of tuning. These four queries find every holder.

**CHECK FIRST** — Run all of §3 before changing any GUC. A single abandoned replication slot or a forgotten idle-in-transaction session defeats every autovacuum setting you own.

## 3.1 Oldest transactions holding back the horizon

*Long-running and idle-in-transaction backends pin the xmin.*

**READ** — `(idle in transaction)` with a large `(in_state_for)` is the classic culprit. `(backend_xmin)` is the transaction id this session pins.

### 3.2 Replication slots pinning xmin / catalog\_xmin

*Inactive or lagging slots are the most common silent bloat cause.*

```
sql
SELECT slot_name, slot_type, active,
       database,
       xmin, catalog_xmin,
       age(xmin) AS xmin_age,
       age(catalog_xmin) AS catalog_xmin_age,
       restart_lsn,
       pg_size_pretty(
         pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
       ) AS retained_wal
FROM   pg_replication_slots
ORDER BY age(COALESCE(xmin, catalog_xmin)) DESC NULLS LAST;
```

**READ** — `(active = false)` with a non-null `(xmin)` = an orphaned slot holding the horizon (and WAL) hostage. Consider `(pg_drop_replication_slot())` once confirmed dead.

### 3.3 Standbys with hot\_standby\_feedback holding xmin

*Feedback from replicas running long queries pins the primary's horizon.*

```
sql
SELECT pid, username, application_name,
       client_addr, state,
       backend_xmin,
       age(backend_xmin) AS feedback_xmin_age,
       replay_lag
FROM   pg_stat_replication
ORDER BY age(backend_xmin) DESC NULLS LAST;
```

**READ** — A large `(feedback_xmin_age)` means a standby's long query is blocking cleanup on the primary. Trace it on the replica's `(pg_stat_activity)`.

### 3.4 Orphaned prepared (two-phase) transactions

*A forgotten PREPARE TRANSACTION pins the horizon forever.*

```
sql
```

```
SELECT gid, database, owner,
       prepared,
       now() - prepared AS prepared_age,
       transaction AS xid
FROM   pg_prepared_xacts
ORDER BY prepared ASC;
```

**READ** — Any row here on a system that doesn't use 2PC is a bug/leak. `ROLLBACK PREPARED '<gid>'` to release. Check `max_prepared_transactions` if you don't expect these.

### 3.5 One-shot horizon summary — who wins?

*Compare all four holders in transaction-age terms; the max age is your real horizon.*

```
sql

SELECT 'backend' AS source, pid::text AS ident, age(backend_xmin) AS xmin_age
FROM   pg_stat_activity          WHERE backend_xmin IS NOT NULL
UNION ALL
SELECT 'repl_slot', slot_name, age(COALESCE(catalog_xmin, xmin))
FROM   pg_replication_slots      WHERE COALESCE(catalog_xmin, xmin) IS NOT NULL
UNION ALL
SELECT 'standby', application_name, age(backend_xmin)
FROM   pg_stat_replication       WHERE backend_xmin IS NOT NULL
UNION ALL
SELECT 'prepared', gid, age(transaction)
FROM   pg_prepared_xacts
ORDER BY xmin_age DESC NULLS LAST
LIMIT 20;
```

**READ** — The top row is whatever is currently defining how far vacuum can clean. Fix that source, and the 'dead but not removable' backlog clears on the next vacuum.

## 4. Transaction-ID wraparound & freeze age

### 4.1 Per-database frozen age vs. wraparound

*Cluster-level wraparound watch. Alert well before the  $2^{31}$  ceiling.*

```
sql

SELECT datname,
       age(datfrozenxid) AS xid_age,
       mxid_age(datminmxid) AS mxid_age,
       round(100.0 * age(datfrozenxid)
            / (2^31)::bigint, 1) AS pct_toward_wrap
FROM   pg_database
ORDER BY age(datfrozenxid) DESC;
```

**READ** — Warnings begin ~40M from wraparound; writes stop with ~3M left. Treat `pct_toward_wrap > 50%` as investigate, `> 75%` as urgent.

### 4.2 Tables closest to forcing an anti-wraparound vacuum

*Which relations will trigger the next forced freeze, and how soon.*

```
sql
```

```

SELECT c.oid::regclass AS table_name,
       age(c.relfrozensxid) AS xid_age,
       mxid_age(c.relminmxid) AS mxid_age,
       current_setting('autovacuum_freeze_max_age')::bigint AS freeze_max_age,
       current_setting('autovacuum_freeze_max_age')::bigint
       - age(c.relfrozensxid) AS xids_until_forced,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS size
FROM   pg_class c
JOIN   pg_namespace n ON n.oid = c.relnamespace
WHERE  c.relkind IN ('r','m','t') -- table, matview, toast
       AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY age(c.relfrozensxid) DESC
LIMIT 40;

```

**READ** — `xids_until_forced` going negative-ish means an anti-wraparound autovacuum is imminent/overdue for that table. Big + old = plan for a long forced freeze.

### 4.3 Effective freeze / failsafe knobs

*The freeze cutoffs that govern when and how hard vacuum freezes.*

```

sql

SELECT name, setting, unit, context, source
FROM   pg_settings
WHERE  name IN (
    'vacuum_freeze_min_age',
    'vacuum_freeze_table_age',
    'vacuum_multixact_freeze_min_age',
    'vacuum_multixact_freeze_table_age',
    'autovacuum_freeze_max_age',
    'autovacuum_multixact_freeze_max_age',
    'vacuum_failsafe_age',           -- PG14+
    'vacuum_multixact_failsafe_age', -- PG14+
    'vacuum_max_eager_freeze_failure_rate') -- PG18
ORDER BY name;

```

**READ** — `source` = 'default' vs 'configuration file' tells you what has actually been overridden. `vacuum_max_eager_freeze_failure_rate` only appears on PG18.

### 4.4 MultiXact member-space pressure

*Wraparound has a second axis: multixact members, not just IDs.*

```

sql

SELECT datname,
       mxid_age(datminmxid) AS mxid_age
FROM   pg_database
ORDER BY mxid_age(datminmxid) DESC;
-- Members exhaustion (SLRU) is separate from ID exhaustion;
-- heavy row-level locking / FK contention drives it.

```

**READ** — High `mxid_age` with heavy `(SELECT ... FOR SHARE)` / FK workloads points at multixact pressure; tune `autovacuum_multixact_freeze_max_age`, not just the xid one.

## 5. Live vacuum progress

### 5.1 pg\_stat\_progress\_vacuum — what each running vacuum is doing

*Phase, heap scan progress, and blocks written.*

```
sql
SELECT p.pid,
       p.datname,
       c.oid::regclass AS table_name,
       p.phase,
       p.heap_blks_total,
       p.heap_blks_scanned,
       p.heap_blks_vacuumed,
       round(100.0 * p.heap_blks_scanned
            / NULLIF(p.heap_blks_total,0), 1) AS pct_scanned,
       p.index_vacuum_count,
       p.max_dead_tuple_bytes,      -- PG17+ (was max_dead_tuples pre-1
       p.dead_tuple_bytes,          -- PG17+
       a.state, a.wait_event
FROM   pg_stat_progress_vacuum p
JOIN   pg_class c ON c.oid = p.relid
LEFT   JOIN pg_stat_activity a ON a.pid = p.pid;
```

**READ** — Pre-PG17 the columns are `(num_dead_tuples) / (max_dead_tuples)` (tuple counts). PG17+ renamed them to `(*_bytes)` because storage is now the radix tree. Drop the columns your version lacks.

### 5.2 Multi-pass detection — the memory-pressure smell

`(index_vacuum_count > 1)` means dead-TID storage overflowed and every index is being scanned again.

```
sql
SELECT c.oid::regclass AS table_name,
       phase,
       index_vacuum_count,
       heap_blks_total, heap_blks_scanned
FROM   pg_stat_progress_vacuum p
JOIN   pg_class c ON c.oid = p.relid
WHERE  index_vacuum_count > 1;
```

**READ** — Common on PG16 and earlier for tables with >~179M dead tuples (the old 1GB TID array). Fix: lower the table's scale factor, or raise `(autovacuum_work_mem)`. On PG17+ this should be rare — if you see it, investigate.

### 5.3 Index-phase progress

*For long index-cleanup phases, see how many indexes remain.*

```
sql
SELECT c.oid::regclass AS table_name,
       phase,
       indexes_total,      -- PG17+
       indexes_processed   -- PG17+
FROM   pg_stat_progress_vacuum p
JOIN   pg_class c ON c.oid = p.relid
WHERE  phase LIKE '%index%';
```

**READ** — `(indexes_total) / (indexes_processed)` are PG17+. Before that you only get `(index_vacuum_count)` (pass number), not per-index progress.



## 5.4 Cumulative time spent vacuuming per table [PG18]

Find tables that soak the most vacuum effort over time.

```
sql

SELECT schemaname, relname,
       vacuum_count, autovacuum_count,
       total_vacuum_time,      -- PG18, milliseconds
       total_autovacuum_time,  -- PG18
       total_analyze_time,     -- PG18
       total_autoanalyze_time  -- PG18
FROM   pg_stat_user_tables
ORDER BY total_autovacuum_time DESC NULLS LAST
LIMIT 30;
```

## 6. Effective settings — global and per-table

### 6.1 All vacuum/autovacuum GUCs with their source

Distinguish defaults from what you actually set, and how to change each.

```
sql

SELECT name, setting, unit,
       context,      -- postmaster=restart, sighup=reload, user/superuser
       source,       -- default / configuration file / override / ...
       boot_val, reset_val
FROM   pg_settings
WHERE  name ~ '(vacuum|freeze|maintenance_work_mem|autovacuum)'
ORDER BY name;
```

**READ** — `context` is your restart-vs-reload cheat column. `source=default` means untouched. `reset_val` is what `SET RESET` would restore to.

### 6.2 Per-table storage-parameter overrides in force

Any table whose *reloptions* change its autovacuum behaviour.

```
sql

SELECT n.nspname AS schema,
       c.relname AS table,
       c.relkind,
       c.reloptions
FROM   pg_class c
JOIN   pg_namespace n ON n.oid = c.relnamespace
WHERE  c.reloptions IS NOT NULL
AND    EXISTS (SELECT 1 FROM unnest(c.reloptions) o
               WHERE o LIKE 'autovacuum%'
                  OR o LIKE 'toast.autovacuum%'
                  OR o LIKE 'vacuum_%'
                  OR o LIKE 'log_autovacuum%')
ORDER BY n.nspname, c.relname;
```

**READ** — These **OVERRIDE** the global GUCs for that table (and TOAST via `toast.*` keys). A table with `autovacuum_enabled=false` shows here — a frequent "why is it bloating" answer.

### 6.3 Tables with autovacuum explicitly disabled

The fastest "gotcha" check.

```
sql

SELECT (n.nspname || '.' || c.relname) AS table_name,
       c.reloptions
FROM   pg_class c
JOIN   pg_namespace n ON n.oid = c.relnamespace
WHERE  c.reloptions::text LIKE '%autovacuum_enabled=false%'
       OR c.reloptions::text LIKE '%autovacuum_enabled=off%';
```

**READ** — Anti-wraparound vacuums still run on these tables regardless, but normal dead-tuple cleanup does not. A very common cause of a single bloated table in an otherwise healthy cluster.

## 7. Blocking, cancellation & starvation

### 7.1 Is a DDL/lock waiter about to cancel an autovacuum?

*Normal autovacuum yields to conflicting lock requests; a loop of DDL can starve a table forever.*

```
sql

SELECT waiting.pid           AS waiting_pid,
       waiting.query         AS waiting_query,
       blocking.pid          AS av_pid,
       blocking.query         AS av_query,
       blocking.backend_type
FROM   pg_stat_activity waiting
JOIN   LATERAL unnest(pg_blocking_pids(waiting.pid)) AS bp(pid) ON true
JOIN   pg_stat_activity blocking ON blocking.pid = bp.pid
WHERE  blocking.backend_type = 'autovacuum worker'
       OR blocking.query ILIKE 'autovacuum:%';
```

**READ** — If a normal (non-wraparound) autovacuum is holding SHARE UPDATE EXCLUSIVE and a conflicting lock request arrives, the autovacuum is cancelled. Repeated DDL/ANALYZE against a hot table can prevent it ever completing.

### 7.2 Anti-wraparound workers (the un-cancellable kind)

*These ignore lock-conflict cancellation — they will block your DDL, not the reverse.*

```
sql

SELECT pid, datname,
       now() - xact_start AS running_for,
       query
FROM   pg_stat_activity
WHERE  query ILIKE '%autovacuum:%'
       AND query ILIKE '%wraparound%';
```

**READ** — ((to prevent wraparound)) in the query text = anti-wraparound. Do NOT kill these casually; killing them just delays the inevitable and they will restart.

### 7.3 Current locks held by autovacuum workers

*See exactly what a worker is locking.*

```
sql
```

```

SELECT l.pid, c.oid::regclass AS relation,
       l.mode, l.granted,
       a.query
FROM   pg_locks l
JOIN   pg_stat_activity a ON a.pid = l.pid
LEFT   JOIN pg_class c ON c.oid = l.relation
WHERE  a.backend_type = 'autovacuum worker'
       OR a.query ILIKE 'autovacuum:%'
ORDER BY l.pid;

```

## 7.4 Worker saturation — are all slots always busy?

*If running workers == max\_workers continuously, the daemon is under-provisioned.*

```

sql

SELECT count(*)                                AS active_workers,
       current_setting('autovacuum_max_workers')::int AS max_workers
FROM   pg_stat_activity
WHERE  backend_type = 'autovacuum worker'
       OR query ILIKE 'autovacuum:%';

```

**READ** — Persistently at max = raise `autovacuum_max_workers` AND raise the cost limit proportionally (the I/O budget is shared across all workers, so more workers without more budget doesn't go faster).

## 8. I/O attribution & AIO

### 8.1 Vacuum I/O via pg\_stat\_io

*Per-context I/O; isolate what vacuum reads/writes/evicts.*

```

sql

SELECT backend_type, context,
       reads, read_bytes,
       writes, write_bytes,
       extends,
       evictions, reuses,
       hits
FROM   pg_stat_io
WHERE  context = 'vacuum'
       OR backend_type = 'autovacuum worker'
ORDER BY backend_type, context;

```

**READ** — `pg_stat_io` is PG16+. High `reuses` in the vacuum context reflects the ring buffer (buffer access strategy) working as intended; high `evictions` can mean the ring is too small — see `vacuum_buffer_usage_limit`.

### 8.2 Column layout differs by version

`read_bytes`/`write_bytes` columns were added later; on PG16 use the `*_time`/op counters instead.

```

sql

-- Portable minimal form (works PG16+):
SELECT backend_type, context, reads, writes, extends, evictions, hits
FROM   pg_stat_io
WHERE  context = 'vacuum';

```

**READ** — If a column errors out, your minor/major version predates it — trim the SELECT list. `read_bytes`/`write_bytes` are PG18.

### 8.3 In-flight async I/O (PG18 AIO)

*Inspect the AIO subsystem while a vacuum-heavy workload runs.*

```
sql

SELECT pid, io_id, io_generation,
       state, operation, off, length,
       target, handle_data_len
FROM   pg_aios
ORDER  BY pid;

-- And confirm the method in use:
SHOW io_method;      -- worker | io_uring | sync   (PG18)
SHOW io_workers;     -- worker count for 'worker' method
SHOW maintenance_io_concurrency; -- vacuum/analyze prefetch depth (default 1)
```

**READ** — `pg_aios` and `io_method` exist only on PG18. Vacuum/analyze benefit from AIO on the READ path (heap & bitmap scans); writes and index scans are not yet AIO-accelerated.

## 9. Live-challenge triage order

A compressed decision path for "autovacuum isn't keeping up", in the order that resolves incidents fastest:

| # | Question                   | Query / action                                                                                                                     |
|---|----------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Is autovacuum on?          | 1.1 — check <code>autovacuum</code> , and 6.3 for a per-table <code>autovacuum_enabled=false</code> .                              |
| 2 | What's running now?        | 1.2 / 5.1 — a worker may already be grinding; note anti-wraparound tags.                                                           |
| 3 | Is the horizon held back?  | §3.5 one-shot summary. If a slot/txn/standby/prepared-xact pins xmin, fix THAT — no GUC will help.                                 |
| 4 | Which tables are overdue?  | 2.1 (vs threshold) + 2.2 (dead %). Confirm with 2.3 pgstattuple if it matters.                                                     |
| 5 | Wraparound risk?           | 4.1 (per-db) + 4.2 (per-table). Anything near <code>freeze_max_age</code> needs a forced freeze plan.                              |
| 6 | Memory / multi-pass?       | 5.2 — <code>index_vacuum_count&gt;1</code> ⇒ lower scale factor (pre-17) or raise <code>work_mem</code> .                          |
| 7 | Being cancelled / starved? | 7.1 (DDL cancelling AV) + 7.4 (all workers busy).                                                                                  |
| 8 | Then tune.                 | Raise <code>cost_limit</code> first, lower <code>cost_delay</code> , per-table scale factors for hot tables, add workers + budget. |

**GOLDEN RULE** — Steps 1-3 explain the large majority of "autovacuum is broken" tickets. Only reach step 8 (tuning) after 1-7 come back clean.

## Appendix. Common remediation snippets

Have these ready — but only after diagnosis. Dynamic (reload) vs restart is noted.

### Make a hot table more aggressive (no restart)

```
sql

ALTER TABLE public.orders SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold   = 1000,
    autovacuum_vacuum_cost_delay   = 0,      -- independent I/O budget
    autovacuum_analyze_scale_factor = 0.02
);
```

### Global cost budget (reload — `pg_reload_conf()`)

```
sql

-- postgresql.conf, then SELECT pg_reload_conf();
autovacuum_vacuum_cost_limit = 2000
autovacuum_vacuum_cost_delay = 2ms    -- 1ms SSD / 0 NVMe
autovacuum_naptime          = 15s
```

### Restart-required knobs (plan a window)

```
sql

autovacuum_max_workers      = 6    -- restart pre-PG18; SIGHUP on PG18
autovacuum_worker_slots     = 16    -- PG18, restart (ceiling for max_)
autovacuum_freeze_max_age   = 400000000
autovacuum_multixact_freeze_max_age = 400000000
io_method                   = worker -- PG18, restart
```

### Release a held horizon (after confirming the source)

```
sql

-- Orphaned replication slot:
SELECT pg_drop_replication_slot('dead_slot_name');
-- Orphaned prepared xact:
ROLLBACK PREPARED 'the_gid';
-- Idle-in-transaction backend:
SELECT pg_terminate_backend(<pid>);
```

### Emergency manual freeze for one table

```
sql

VACUUM (FREEZE, VERBOSE) public.big_cold_table;
-- PG16+ throttle the buffer footprint:
VACUUM (FREEZE, BUFFER_USAGE_LIMIT '256kB') public.big_cold_table;
```

**CAVEAT** — Version-gated features: `pg_stat_io` (16+), TID-store progress columns and `indexes_total/processed` (17+), `pg_aio` / `io_method` / `autovacuum_vacuum_max_threshold` / `total_*_time` / SIGHUP `max_workers` (18). Trim columns that error on older minors.